Homework #2

CSE 546: Machine Learning
Tyler Shakibai

Collaborators: Jake Murray, Peter Sushko

Conceptual Questions

A1. The answers to these questions should be answerable without referring to external materials. Briefly justify your answers with a few words.

a. *[2 points]* Compared to L2 norm penalty, explain why a L1 norm penalty is more likely to result in sparsity (a larger number of 0s) in the weight vector.

The L2 norm penalty adds a ball-shaped region constraint and finds more dense solutions. The L1 norm penalty adds a diamond-shaped region constraint which promotes sparsity because the intersection with concentric level set circles is likely to be a corner of the diamond where one component is 0. Since the L1 norm is pointy and the L2 norm is smooth, the L1 norm penalty is more likely to result in sparse solutions.

b. *[2 points]* In at most one sentence each, state one possible upside and one possible downside of using the following regularizer: $\left(\sum_i |w_i|^{0.5}\right)$.

This regularizer is more pointy and promotes even sparser solutions than the L1 norm penalty. However, this regularizer is non-convex.

c. *[2 points]* True or False: If the step-size for gradient descent is too large, it may not converge.

True. If the step-size for gradient descent is too large, it may overshoot the desired minimum, causing it to oscillate or diverge.

d. *[2 points]* In at most one sentence each, state one possible advantage of SGD over GD (gradient descent), and one possible disadvantage of SGD relative to GD.

SGD is much less computationally expensive per iteration compared to GD. SGD can be less accurate relative to GD due to the noise in the mini-batches and may require more iterations to achieve the same error.

e. *[2 points]* Why is it necessary to apply the gradient descent algorithm on logistic regression but not linear regression?

Unlike linear regression, there is no closed-form solution for logistic regression. Thus, we apply gradient descent to find optimal weights.

Convexity and Norms

A2. A *norm* $\|\cdot\|$ over $\mathbb{R}^n$ is defined by the properties: (i) non-negativity: $\|x\| \geq 0$ for all $x \in \mathbb{R}^n$ with equality if and only if $x = 0$, (ii) absolute scalability: $\|a\,x\| = |a|\,\|x\|$ for all $a \in \mathbb{R}$ and $x \in \mathbb{R}^n$, (iii) triangle inequality: $\|x + y\| \leq \|x\| + \|y\|$ for all $x, y \in \mathbb{R}^n$.

a. *[3 points]* Show that $f(x) = (\sum_{i=1}^n |x_i|)$ is a norm. (Hint: for (iii), begin by showing that $|a+b| \leq |a|+|b|$ for all $a, b \in \mathbb{R}$.)

(i) Non-negativity:
$\forall i, x_i \in \mathbb{R}$ and $|x_i| \geq 0$:

$$f(x) = \|x\| = \sum_{i=1}^n |x_i| \geq 0$$

This holds for all $x \in \mathbb{R}^n$ with $|x_i| = 0$ iff $x_i = 0$, so $f(x) = 0$ if and only if $x = 0$.

(ii) Absolute scalability:
$\forall a \in \mathbb{R}, x \in \mathbb{R}^n$:

$$f(ax) = \|ax\| = \sum_{i=1}^n |ax_i| = \sum_{i=1}^n |a||x_i| = |a| \sum_{i=1}^n |x_i| = |a|\|x\| = |a|f(x)$$

(iii) Triangle inequality:
First, we show that $|a + b| \leq |a| + |b|$

$$|a+b|^2 = a^2 + 2ab + b^2 \leq |a|^2 + 2|a||b| + |b|^2 = (|a| + |b|)^2$$

$$\implies |a + b| \leq |a| + |b| \text{ since } |a + b| \geq 0, |a| + |b| \geq 0$$

Then we have that

$$f(x+y) = \|x+y\| = \sum_{i=1}^n |x_i + y_i| \leq \sum_{i=1}^n (|x_i| + |y_i|) = \sum_{i=1}^n |x_i| + \sum_{i=1}^n |y_i| = \|x\| + \|y\| = f(x) + f(y)$$

hence

$$f(x + y) \leq f(x) + f(y)$$

Therefore $f(x) = \sum_{i=1}^n |x_i|$ is a norm.

b. *[2 points]* Show that $g(x) = \left(\sum_{i=1}^n |x_i|^{1/2}\right)^2$ is not a norm. (Hint: it suffices to find two points in $n = 2$ dimensions such that the triangle inequality does not hold.)

Consider the points in $\mathbb{R}^2 : x = [0, c]^T$, $y = [c, 0]^T$ where $c > 0$ is some arbitrary constant.

$$g(x + y) = 4c$$

$$g(x) + g(y) = 2c$$

We thus have

$$g(x + y) > g(x) + g(y)$$

Since it does not satisfy the triangle inequality, $g(x) = \left(\sum_{i=1}^n |x_i|^{1/2}\right)^2$ is not a norm.

Context: norms are often used in regularization to encourage specific behaviors of solutions. If we define $\|x\|_p := (\sum_{i=1}^n |x_i|^p)^{1/p}$ then one can show that $\|x\|_p$ is a norm for all $p \geq 1$. The important cases of $p = 2$ and $p = 1$ correspond to the penalty for ridge regression and the lasso, respectively.

B1. *[6 points]* For any $x \in \mathbb{R}^n$, define the following norms: $\|x\|_1 = \sum_{i=1}^n |x_i|$, $\|x\|_2 = \sqrt{\sum_{i=1}^n |x_i|^2}$, $\|x\|_\infty := \lim_{p\to\infty} \|x\|_p = \max_{i=1,\ldots,n} |x_i|$. Show that $\|x\|_\infty \leq \|x\|_2 \leq \|x\|_1$.

$$||x||_1^2 = \sum_{i=1}^N |x_i|^2 + 2\sum_{i\neq j}^N |x_i||x_j| \geq \sum_{i=1}^N |x_i|^2 = ||x||_2^2$$

Since $\|x\|_2 \geq 0, \|x\|_1 \geq 0$, we have

$$||x||_2 \leq ||x||_1$$

Using

$$|x_i| \geq 0 \Rightarrow \max_i(|x_i|^2) = (\max_i |x_i|)^2$$

we have that

$$\|x\|_\infty^2 = (\max_{i=1,\ldots,n} |x_i|)^2 = (\max_i |x_i|)^2 = \max_i(|x_i|^2) \leq \sum_{i=1}^N |x_i|^2 = \|x\|_2^2$$

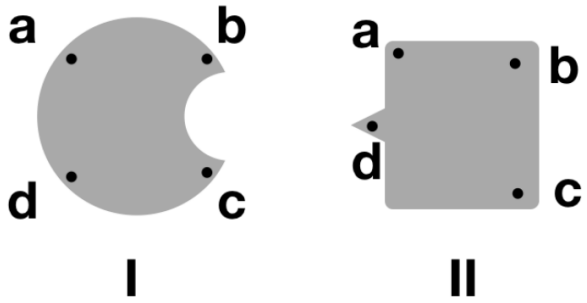
Since $\|x\|_2 \geq 0, \|x\|_\infty \geq 0$, we have

$$\|x\|_\infty^2 \leq ||x||_2$$

Therefore

$$\|x\|_\infty^2 \leq ||x||_2 \leq ||x||_1$$

A3. *[2 points]* A set $A \subseteq \mathbb{R}^n$ is *convex* if $\lambda x + (1-\lambda)y \in A$ for all $x, y \in A$ and $\lambda \in [0, 1]$. For each of the grey-shaded sets below (I-II), state whether each one is convex, or state why it is not convex using any of the points a, b, c, d in your answer.

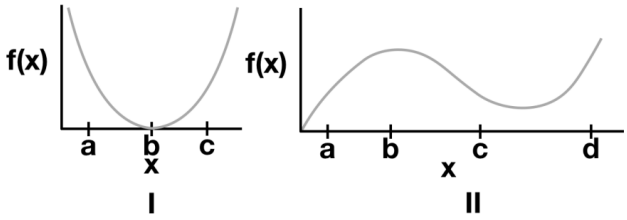


I is not convex. The line segment connecting b and c exits the set.
II is not convex. The line segment connecting a and d exits the set.

A4. *[2 points]* We say a function $f : \mathbb{R}^d \to \mathbb{R}$ is convex on a set A if $f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$ for all $x, y \in A$ and $\lambda \in [0,1]$. For each of the functions shown below (I-II), state whether each is convex on the specified interval, or state why not with a counterexample using any of the points a, b, c, d in your answer.

a. Function in panel I on $[a, c]$

b. Function in panel II on $[a, d]$



I is convex. $f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$, $\lambda \in [0,1]$ holds and any line segment connecting two points of the function on $[a, c]$ lies above the function.
II is not convex. The line segments connecting $f(a)$ and $f(c)$ or $f(a)$ and $f(d)$ lie below the function.

B2. For $i = 1, \ldots, n$ let $\ell_i(w)$ be convex functions over $w \in \mathbb{R}^d$ (e.g., $\ell_i(w) = (y_i - w^\top x_i)^2$), $\|\cdot\|$ is any norm, and $\lambda > 0$.

a. *[3 points]* Show that

$$\sum_{i=1}^{n} \ell_i(w) + \lambda \|w\|$$

is convex over $w \in \mathbb{R}^d$ (Hint: Show that if f, g are convex functions, then $f(x) + g(x)$ is also convex.)

We start by showing that, for any $\lambda \in [0,1]$, if $f(x)$, $g(x)$ are convex functions $\implies f(x) + g(x)$ is also convex.

$$f(\lambda x + (1-\lambda)y) + g(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y) + \lambda g(x) + (1-\lambda)g(y)$$

$$\lambda f(x) + (1-\lambda)f(y) + \lambda g(x) + (1-\lambda)g(y) = \lambda(f(x) + g(x)) + (1-\lambda)(f(y) + g(y))$$

We now show that sum of n convex functions $f_i(x)$ is also convex. $f_1(x) + f_2(x)$ is convex, which implies the convexity of $(f_1(x) + f_2(x)) + f_3(x)$ by our previous result. At every step k, we know that $\sum_{i=1}^{k} f_i(x)$ is convex and $f_{k+1}(x)$ is also convex, thus $\sum_{i=1}^{k+1} f_i(x)$ is convex. By induction, it follows that $\sum_{i=1}^{n} f_i(x)$ is convex.
Next, we show that $\lambda\|w\|$ is convex because $\|w\|$ is convex. For any $\gamma \in [0,1]$, we have that

$$\lambda\|\gamma x + (1-\gamma)y\| \leq \lambda(\gamma\|x\| + (1-\gamma)\|y\|) = \gamma\lambda\|x\| + (1-\gamma)\lambda\|y\|$$

Since $l_i(w)$ is convex $\Rightarrow \sum_{i=1}^{n} l_i(w)$ is convex and $\|w\|$ is convex $\Rightarrow \lambda\|w\|$ is also convex. Therefore

$$\sum_{i=1}^{n} l_i(w) + \lambda\|w\|$$

is convex as it is the sum of two convex functions.

b. *[1 point]* Explain in one sentence why we prefer to use loss functions and regularized loss functions that are convex.

Convexity guarantees that any local minimum is a global minimum.

Lasso on a Real Dataset

Given $\lambda > 0$ and data $\left(x_i, y_i\right)_{i=1}^n$, the Lasso is the problem of solving

$$\arg \min_{w \in \mathbb{R}^d, b \in \mathbb{R}} \sum_{i=1}^n (x_i^T w + b - y_i)^2 + \lambda \sum_{j=1}^d |w_j|$$

where λ is a regularization parameter. For the programming part of this homework, you will implement the iterative shrinkage thresholding algorithm shown in Algorithm 1 to solve the Lasso problem in `ISTA.py`. This is a variant of the subgradient descent method and a more detailed discussion can be found in this slides. You may use common computing packages (such as `numpy` or `scipy`), but do not use an existing Lasso solver (e.g., of `scikit-learn`).

Algorithm 1: Iterative Shrinkage Thresholding Algorithm for Lasso

Input: Step size η
while *not converged* **do**
  $b' \leftarrow b - 2\eta \sum_{i=1}^n (x_i^T w + b - y_i)$
  for $k \in \{1, 2, \cdots d\}$ **do**
    $w'_k \leftarrow w_k - 2\eta \sum_{i=1}^n x_{i,k}(x_i^T w + b - y_i)$
    $$w'_k \leftarrow \begin{cases} w'_k + 2\eta\lambda & w'_k < -2\eta\lambda \\ 0 & w'_k \in [-2\eta\lambda, 2\eta\lambda] \\ w'_k - 2\eta\lambda & w'_k > 2\eta\lambda \end{cases}$$
  end
  $b \leftarrow b'$, $w \leftarrow w'$
end

Before you get started, the following hints may be useful:

- Wherever possible, use matrix libraries for matrix operations (not `for` loops). This especially applies to computing the updates for w. While we wrote the algorithm above with a `for` loop for clarity, you should be able to replace this loop using equivalent matrix/vector operations in your code (e.g., `numpy` functions).
- As a sanity check, ensure the objective value is nonincreasing with each step.
- It is up to you to decide on a suitable stopping condition. A common criteria is to stop when no element of w changes by more than some small δ during an iteration. If you need your algorithm to run faster, an easy place to start is to loosen this condition.
- You will need to solve the Lasso on the same dataset for many values of λ. This is called a regularization path. One way to do this efficiently is to start at a large λ, and then for each consecutive solution, initialize the algorithm with the previous solution, decreasing λ by a constant ratio (e.g., by a factor of 2).
- The smallest value of λ for which the solution $\widehat{w}$ is entirely zero is given by

$$\lambda_{max} = \max_{k=1,\ldots,d} 2 \left| \sum_{i=1}^n x_{i,k} \left(y_i - \left(\frac{1}{n} \sum_{j=1}^n y_j \right) \right) \right| \tag{1}$$

  This is helpful for choosing the first λ in a regularization path.

A5. We will first try out your solver with some synthetic data. A benefit of the Lasso is that if we believe many features are irrelevant for predicting y, the Lasso can be used to enforce a sparse solution, effectively differentiating between the relevant and irrelevant features. Suppose that $x \in \mathbb{R}^d, y \in \mathbb{R}, k < d$, and data are generated independently according to the model $y_i = w^T x_i + \epsilon_i$ where

$$w_j = \begin{cases} j/k & \text{if } j \in \{1, \ldots, k\} \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

and $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ is noise (note that in the model above $b = 0$). We can see from Equation (2) that since $k < d$ and $w_j = 0$ for $j > k$, the features $k + 1$ through d are irrelevant for predicting y.
Generate a dataset using this model with $n = 500, d = 1000, k = 100$, and $\sigma = 1$. You should generate the dataset such that each $\epsilon_i \sim \mathcal{N}(0, 1)$, and y_i is generated as specified above. You are free to choose a distribution from which the x's are drawn, but make sure standardize the x's before running your experiments.

a. *[10 points]* With your synthetic data, solve multiple Lasso problems on a regularization path, starting at λ_{max} where no features are selected (see Equation (1)) and decreasing λ by a constant ratio (e.g., 2) until nearly all the features are chosen. In plot 1, plot the number of non-zeros as a function of λ on the x-axis (Tip: use `plt.xscale('log')`).

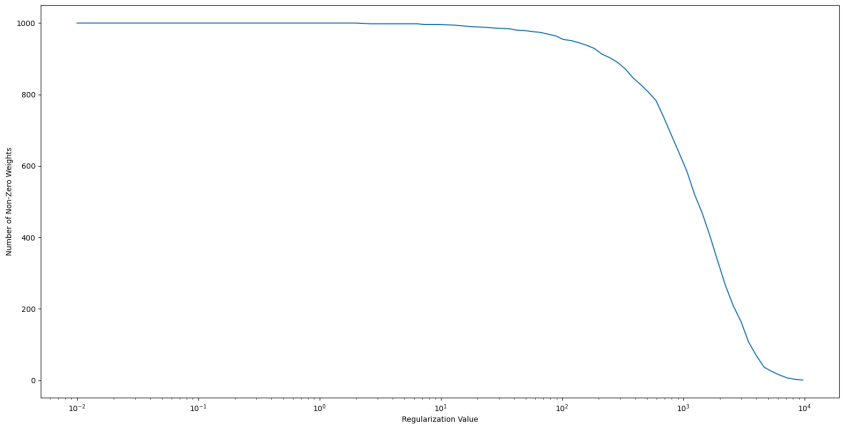


Figure 1: Non-zero weights versus λ.

b. *[10 points]* For each value of λ tried, record values for false discovery rate (FDR) (number of incorrect nonzeros in $\widehat{w}$/total number of nonzeros in $\widehat{w}$) and true positive rate (TPR) (number of correct nonzeros in $\widehat{w}$/k). Note: for each j, $\widehat{w}_j$ is an incorrect nonzero if and only if $\widehat{w}_j \neq 0$ while $w_j = 0$. In plot 2, plot these values with the x-axis as FDR, and the y-axis as TPR.

Note that in an ideal situation we would have an (FDR,TPR) pair in the upper left corner. We can always trivially achieve $(0, 0)$ and $(\frac{d-k}{d}, 1)$.

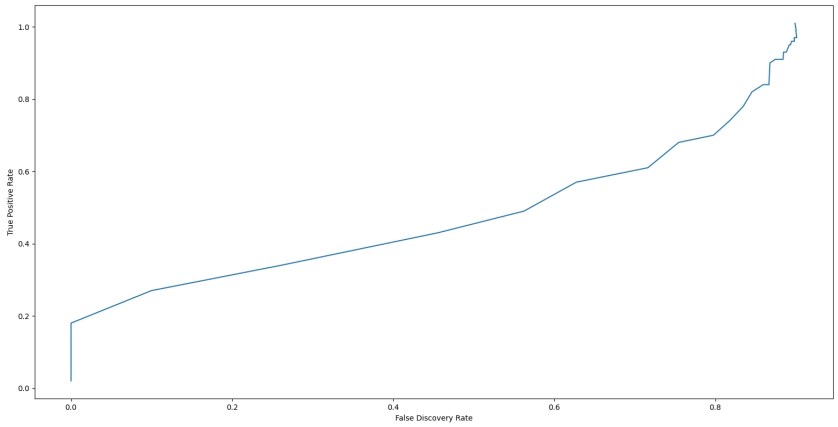


Figure 2: True positive rate versus false discovery rate.

c. *[5 points]* Comment on the effect of λ in these two plots in 1-2 sentences.

A larger λ causes there to be more zero values in the weight vector. A smaller λ causes there to be more non-zero values in the weight vector will be non-zero. Thus, increasing λ results in a more sparse solution. In Figure 2, we see that too large of a λ leads to a zero solution, which has poor performance. If λ is too small, then we have a high false discovery rate.

```
@problem.tag("hw2-A")
def step(
    X: np.ndarray, y: np.ndarray, weight: np.ndarray, bias: float, _lambda: float, eta:
) -> Tuple[np.ndarray, float]:
    """Single step in ISTA algorithm.
    It should update every entry in weight, and then return an updated version of weight

    Args:
        X (np.ndarray): An (n x d) matrix, with n observations each with d features.
        y (np.ndarray): An (n, ) array, with n observations of targets.
        weight (np.ndarray): An (d,) array. Weight returned from the step before.
        bias (float): Bias returned from the step before.
        _lambda (float): Regularization constant. Determines when weight is updated to
        eta (float): Step-size. Determines how far the ISTA iteration moves for each ste

    Returns:
        Tuple[np.ndarray, float]: Tuple with 2 entries. First represents updated weight

    """
    errVec = ((X @ weight) + bias) - y
    errSum = np.sum(errVec)
    bias -= 2 * eta * errSum
    weight -= 2 * eta * (X.T @ errVec)
    regStep = 2 * eta * _lambda

    argZero = np.logical_and(weight > -regStep, weight < regStep)
    weight[weight < -regStep] = weight[weight < -regStep] + regStep
    weight[weight > regStep] = weight[weight > regStep] - regStep
    weight[argZero] = 0
```

```
    return weight, bias


@problem.tag("hw2-A")
def loss(
    X: np.ndarray, y: np.ndarray, weight: np.ndarray, bias: float, _lambda: float
) -> float:
    """L-1 (Lasso) regularized MSE loss.

    Args:
        X (np.ndarray): An (n x d) matrix, with n observations each with d features.
        y (np.ndarray): An (n, ) array, with n observations of targets.
        weight (np.ndarray): An (d,) array. Currently predicted weights.
        bias (float): Currently predicted bias.
        _lambda (float): Regularization constant. Should be used along with L1 norm of

    Returns:
        float: value of the loss function
    """
    return np.sum((((X @ weight) + bias) - y)**2) + _lambda * np.sum(np.abs(weight))


@problem.tag("hw2-A", start_line=5)
def train(
    X: np.ndarray,
    y: np.ndarray,
    _lambda: float = 0.01,
    eta: float = 0.001,
    convergence_delta: float = 1e-4,
    start_weight: np.ndarray = None,
    start_bias: float = None
) -> Tuple[np.ndarray, float]:
    """Trains a model and returns predicted weight and bias.

    Args:
        X (np.ndarray): An (n x d) matrix, with n observations each with d features.
        y (np.ndarray): An (n, ) array, with n observations of targets.
        _lambda (float): Regularization constant. Should be used for both step and loss.
        eta (float): Step size.
        convergence_delta (float, optional): Defines when to stop training algorithm.
            The smaller the value the longer algorithm will train.
            Defaults to 1e-4.
        start_weight (np.ndarray, optional): Weight for hot-starting model.
            If None, defaults to array of zeros. Defaults to None.
            It can be useful when testing for multiple values of lambda.
        start_bias (np.ndarray, optional): Bias for hot-starting model.
            If None, defaults to zero. Defaults to None.
            It can be useful when testing for multiple values of lambda.

    Returns:
        Tuple[np.ndarray, float]: A tuple with first item being array of shape (d,) rep
            and second item being a float representing the bias.

    Note:
        - You will have to keep an old copy of weights for convergence criterion functio
```

```
            Please use `np.copy(...)` function, since numpy might sometimes copy by refe
            instead of by value leading to bugs.
        - You might wonder why do we also return bias here, if we don't need it for this
            There are two reasons for it:
                - Model is fully specified only with bias and weight.
                    Otherwise you would not be able to make predictions.
                    Training function that does not return a fully usable model is just
                - You will use bias in next problem.
    """
    if start_weight is None:
        start_weight = np.zeros(X.shape[1])
        start_bias = 0
    currW: Optional[np.ndarray] = start_weight
    currB: Optional[np.ndarray] = start_bias

    flag = True
    while flag:
        nextW, nextB = step(X, y, currW, currB, _lambda, eta)
        flag = not convergence_criterion(nextW, currW, nextB, currB, convergence_delta=
        currW = nextW.copy()
        currB = nextW.copy()

    return nextW, nextB


@problem.tag("hw2-A")
def convergence_criterion(
    weight: np.ndarray, old_w: np.ndarray, bias: float, old_b: float, convergence_delta:
) -> bool:
    """Function determining whether weight has converged or not.
    It should calculate the maximum absolute change between weight and old_w vector, and

    Args:
        weight (np.ndarray): Weight from current iteration of coordinate gradient descen
        old_w (np.ndarray): Weight from previous iteration of coordinate gradient descen
        convergence_delta (float): Aggressiveness of the check.

    Returns:
        bool: False, if weight has not converged yet. True otherwise.
    """
    if np.max(np.abs(old_w - weight)) > convergence_delta:
        convergence = False
    else:
        convergence = True

    return convergence


@problem.tag("hw2-A")
def main():
    """
    Use all of the functions above to make plots.
    """
    n = 500
    d = 1000
```

```
k = 100
x = np.random.chisquare(100, (n, d))
w = np.zeros(d)
w[:100] = np.linspace(1, 100, 100)/k

y = (x @ w) + np.random.normal(0, 1, n)
standX = (x - x.mean(axis=0))/x.std(axis=0)
_lambda_max = np.max(2 * np.abs(standX.T @ (y - y.mean())))

dense_weights = []
truPosRate = []
falDiscRate = []
regVals = []
for lam in np.geomspace(0.01, _lambda_max, 100):
    weight, bias = train(standX, y, lam)
    if np.count_nonzero(weight) != 0:
        dense_weights.append(np.count_nonzero(weight))
        truPosRate.append(np.count_nonzero(weight[:k+1])/k)
        falDiscRate.append(np.count_nonzero(weight[k+1:])/np.count_nonzero(weight))
        regVals.append(lam)

fig, ax = plt.subplots()
ax.semilogx(regVals, dense_weights)
ax.set_xlabel('Regularization Value')
ax.set_ylabel('Number of Non-Zero Weights')
plt.show()

fig, ax = plt.subplots()
ax.plot(falDiscRate, truPosRate)
ax.set_xlabel('False Discovery Rate')
ax.set_ylabel('True Positive Rate')
plt.show()
```

A6. We'll now put the Lasso to work on some real data in `crime_data_lasso.py`. . Download the training data set "crime-train.txt" and the test data set "crime-test.txt" from the website. Store your data in your working directory, ensure you have the `pandas` library for Python installed, and read in the files with:

```
import pandas as pd
df_train = pd.read_table("crime-train.txt")
df_test = pd.read_table("crime-test.txt")
```

This stores the data as Pandas `DataFrame` objects. `DataFrame`s are similar to Numpy `array`s but more flexible; unlike `array`s, `DataFrame`s store row and column indices along with the values of the data. Each column of a `DataFrame` can also store data of a different type (here, all data are floats). Here are a few commands that will get you working with Pandas for this assignment:

```
df.head()                    # Print the first few lines of DataFrame df.
df.index                     # Get the row indices for df.
df.columns                   # Get the column indices.
df[''foo'']                 # Return the column named ''foo''.
df.drop(''foo'', axis = 1)  # Return all columns except ''foo''.
df.values                    # Return the values as a Numpy array.
df[''foo''].values          # Grab column foo and convert to Numpy array.
df.iloc[:3,:3]               # Use numerical indices (like Numpy) to get 3 rows and cols.
```

The data consist of local crime statistics for 1,994 US communities. The response y is the rate of violent crimes reported per capita in a community. The name of the response variable is `ViolentCrimesPerPop`, and it is held in the first column of `df_train` and `df_test`. There are 95 features. These features include many variables. Some features are the consequence of complex political processes, such as the size of the police force and other systemic and historical factors. Others are demographic characteristics of the community, including self-reported statistics about race, age, education, and employment drawn from Census reports.

The goals of this problem are threefold: (i) to encourage you to think about how data collection processes affect the resulting model trained from that data; (ii) to encourage you to think deeply about models you might train and how they might be misused; and (iii) to see how Lasso encourages sparsity of linear models in settings where d is large relative to n. **We emphasize that training a model on this dataset can suggest a degree of correlation between a community's demographics and the rate at which a community experiences and reports violent crime. We strongly encourage students to consider why these correlations may or may not hold more generally, whether correlations might result from a common cause, and what issues can result in misinterpreting what a model can explain.**

The dataset is split into a training and test set with 1,595 and 399 entries, respectively[1]. We will use this training set to fit a model to predict the crime rate in new communities and evaluate model performance on the test set. As there are a considerable number of input variables and fairly few training observations, overfitting is a serious issue. In order to avoid this, use the coordinate descent Lasso algorithm implemented in the previous problem.

a. *[4 points]* Read the documentation for the originalcversion of this dataset: `http://archive.ics.uci.edu/ml/datasets/communities+and+crime`. Report 3 features included in this dataset for which historical *policy* choices in the US would lead to variability in these features. As an example, the *number of police* in a community is often the consequence of decisions made by governing bodies, elections, and amount of tax revenue available to decision makers.

   The operating budget for a police department is likely affected by historical policies that dictate their capabilities. The number of drugs seized by police units is also a function of the policies that give them the authority to do so. The number of officers assigned to special drug units is also likely affected by historical policies.

b. *[4 points]* Before you train a model, describe 3 features in the dataset which might, if found to have nonzero weight in model, be interpreted as *reasons* for higher levels of violent crime, but which might actually be a *result* rather than (or in addition to being) the cause of this violence.

   Features relating to requests for police presence are likely the result of violent crime but could be mistaken as the cause. Features such as LemasTotalReq (total requests for police), LemasTotReqPerPop (total requests for police per 100K population), and PolicReqPerOfficg (total requests for police per police officer) are all examples of features which might, if found to have nonzero weight in the model, be incorrectly interpreted as causes for higher levels of violent crime.

Now, we will run the Lasso solver. Begin with $\lambda = \lambda_{\max}$ defined in Equation (1). Initialize all weights to 0. Then, reduce λ by a factor of 2 and run again, but this time initialize $\hat{w}$ from your $\lambda = \lambda_{\max}$ solution as your initial weights, as described above. Continue the process of reducing λ by a factor of 2 until $\lambda < 0.01$. For all plots use a log-scale for the λ dimension (Tip: use `plt.xscale('log')`).

c. *[4 points]* Plot the number of nonzero weights of each solution as a function of λ.

[1] The features have been standardized to have mean 0 and variance 1.

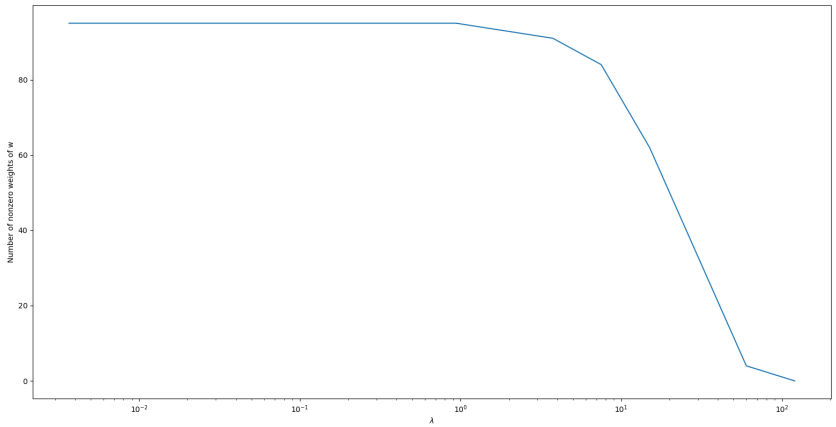


Figure 3: Non-zero weights versus λ.

d. *[4 points]* Plot the regularization paths (in one plot) for the coefficients for input variables `agePct12t29`, `pctWSocSec`, `pctUrban`, `agePct65up`, and `householdsize`.

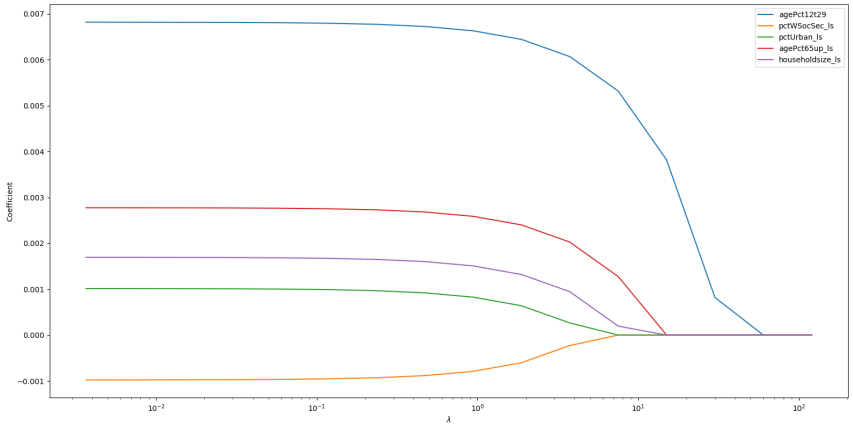


Figure 4: Regularization paths of coefficients.

e. *[4 points]* On one plot, plot the mean squared error on the training and test data as a function of λ.

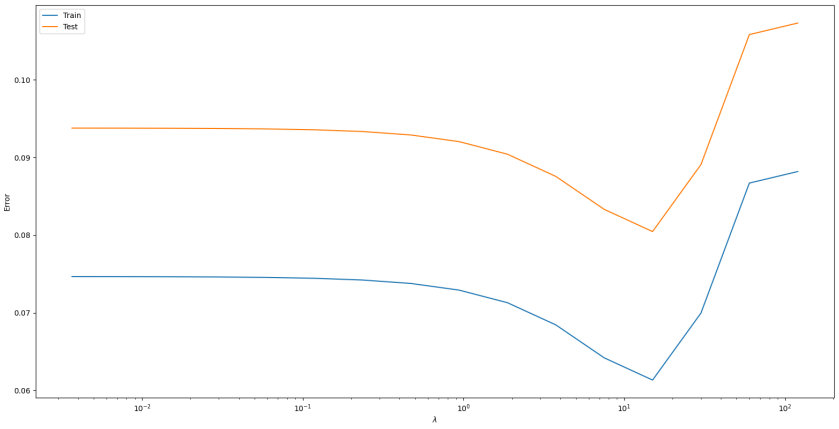


Figure 5: Mean Squared Error versus λ.

f. *[4 points]* Sometimes a larger value of λ performs nearly as well as a smaller value, but a larger value will select fewer variables and perhaps be more interpretable. Inspect the weights $\hat{w}$ for $\lambda = 30$. Which feature had the largest (most positive) Lasso coefficient? What about the most negative? Discuss briefly.

The feature with the largest Lasso coefficient is PctIlleg (percentage of kids born to never married), which is 0.00724164. The smallest is PctKids2Par (percentage of kids in family housing with two parents), which is -0.00757012. PctIlleg has the highest weight, and indicates that kids born to parents who were never married are predicted to be associated with high violent crimes per population. This could be the case because children with this background do not benefit from the same amount of stability as kids in family housing with two parents (which has a negative weight).

g. *[4 points]* Suppose there was a large negative weight on `agePct65up` and upon seeing this result, a politician suggests policies that encourage people over the age of 65 to move to high crime areas in an effort to reduce crime. What is the (statistical) flaw in this line of reasoning? (Hint: fire trucks are often seen around burning buildings, do fire trucks cause fire?)

A large negative weight on agePct65up implies there is a high correlation between agePct65up and ViolentCrimesPerPop. However, agePct65up might not be the cause of the lower crime rate. If an area with a high crime rate already, it is possible that people over 65 would be inclined to move into areas with lower crime rates.

```
@problem.tag("hw2-A", start_line=3)
def main():
    # df_train and df_test are pandas dataframes.
    # Make sure you split them into observations and targets
    df_train, df_test = load_dataset("crime")
    x_train = df_train.drop('ViolentCrimesPerPop', axis = 1)
    y_train = df_train['ViolentCrimesPerPop']
    x = np.asarray(df_test.drop('ViolentCrimesPerPop', axis = 1))
    y = np.asarray(df_test['ViolentCrimesPerPop'])
    x_test = np.asarray(df_test.drop('ViolentCrimesPerPop', axis = 1))
    y_test = np.asarray(df_test['ViolentCrimesPerPop'])

    lambdaMax = np.max(2 * np.sum(x.T * (y - np.mean(y)), axis=0))
```

```
w = np.zeros((x.shape[1],))
lambdas = [lambdaMax/(2**i) for i in range(16)]

noNumZeros, train_mse, test_mse, agePct12t29_ls, pctWSocSec_ls, pctUrban_ls, agePct6
for lam in lambdas:
    w = train(x, y, lam, 0.0001, None)[0]
    b = train(x, y, lam, 0.0001, None)[1]
    notZeros = np.count_nonzero(w)
    noNumZeros.append(notZeros)

    y_preds = x.dot(w) + b
    y_test_preds = x_test.dot(w) + b
    train_mse_val = np.mean((y - y_preds)**2 - b)
    test_mse_val = np.mean((y_test - y_test_preds)**2)
    train_mse.append(train_mse_val)
    test_mse.append(test_mse_val)

    agePct12t29_ls.append(w[0])
    pctWSocSec_ls.append(w[1])
    pctUrban_ls.append(w[2])
    agePct65up_ls.append(w[3])
    householdsize_ls.append(w[4])

plt.figure(figsize = (10, 5))
plt.plot(lambdas, noNumZeros)
plt.xscale('log')
plt.xlabel("$\lambda$")
plt.ylabel("Number of nonzero weights of w")
plt.show()

plt.figure(figsize = (10, 5))
plt.plot(lambdas, agePct12t29_ls, label='agePct12t29')
plt.plot(lambdas, pctWSocSec_ls, label='pctWSocSec_ls')
plt.plot(lambdas, pctUrban_ls, label='pctUrban_ls')
plt.plot(lambdas, agePct65up_ls, label='agePct65up_ls')
plt.plot(lambdas, householdsize_ls, label='householdsize_ls')
plt.xscale('log')
plt.xlabel("$\lambda$")
plt.ylabel('Coefficient')
plt.legend()
plt.show()

plt.figure(figsize = (10, 5))
plt.plot(lambdas, train_mse, label = 'Train')
plt.plot(lambdas, test_mse, label = 'Test')
plt.xscale('log')
plt.xlabel("$\lambda$")
plt.ylabel("Error")
plt.legend()
plt.show()

w = train(x, y, 30, 1e-4, None)[0]
print(x_train.columns[np.argmax(w)])
print(np.max(w))
print(x_train.columns[np.argmin(w)])
```

```
print(np.min(w))
```

Logistic Regression

A7. Here we consider the MNIST dataset, but for binary classification. Specifically, the task is to determine whether a digit is a 2 or 7. Here, let $Y = 1$ for all the "7" digits in the dataset, and use $Y = -1$ for "2". We will use regularized logistic regression. Given a binary classification dataset $\{(x_i, y_i)\}_{i=1}^n$ for $x_i \in \mathbb{R}^d$ and $y_i \in \{-1, 1\}$ we showed in class that the regularized negative log likelihood objective function can be written as

$$J(w, b) = \frac{1}{n} \sum_{i=1}^{n} \log(1 + \exp(-y_i(b + x_i^T w))) + \lambda ||w||_2^2$$

Note that the offset term b is not regularized. For all experiments, use $\lambda = 10^{-1}$. Let $\mu_i(w, b) = \frac{1}{1+\exp(-y_i(b+x_i^T w))}$.

a. *[8 points]* Derive the gradients $\nabla_w J(w, b)$, $\nabla_b J(w, b)$ and give your answers in terms of $\mu_i(w, b)$ (your answers should not contain exponentials).

We start by rewriting

$$\mu_i(w, b) = \frac{1}{1 + \exp(-y_i(b + x_i^T w))}$$

as

$$e^{-y_i(b+x_i^T w)} = \frac{1}{\mu_i(w, b)} - 1$$

We are then able to derive the gradients.

$$\nabla_w J(w, b) = \frac{1}{n} \sum_{i=1}^{n} \nabla_w \log(1+\exp(-y_i(b+x_i^T w))) + \nabla_w \lambda \|w\|^2 = \frac{1}{n} \sum_{i=1}^{n} (-y_i)\mu_i(w, b)(\frac{1}{\mu_i(w, b)} - 1)x_i + 2\lambda w$$

$$\nabla_w J(w, b) = \frac{1}{n} \sum_{i=1}^{n} y_i(\mu_i(w, b) - 1)x_i + 2\lambda w$$

and

$$\nabla_b J(w, b) = \frac{1}{n} \sum_{i=1}^{n} \nabla_b \log(1 + \exp(-y_i(b + x_i^T w))) + \nabla_b \lambda \|w\|^2 = \frac{1}{n} \sum_{i=1}^{n} (-y_i)\mu_i(w, b)(\frac{1}{\mu_i(w, b)} - 1)$$

$$\nabla_b J(w, b) = \frac{1}{n} \sum_{i=1}^{n} y_i(\mu_i(w, b) - 1)$$

b. *[8 points]* Implement gradient descent with an initial iterate of all zeros. Try several values of step sizes to find one that appears to make convergence on the training set as fast as possible. Run until you feel you are near to convergence.

(i) For both the training set and the test, plot $J(w, b)$ as a function of the iteration number (and show both curves on the same plot).

(ii) For both the training set and the test, classify the points according to the rule $\text{sign}(b + x_i^T w)$ and plot the misclassification error as a function of the iteration number (and show both curves on the same plot).

Reminder: Make sure you are only using the test set for evaluation (not for training).

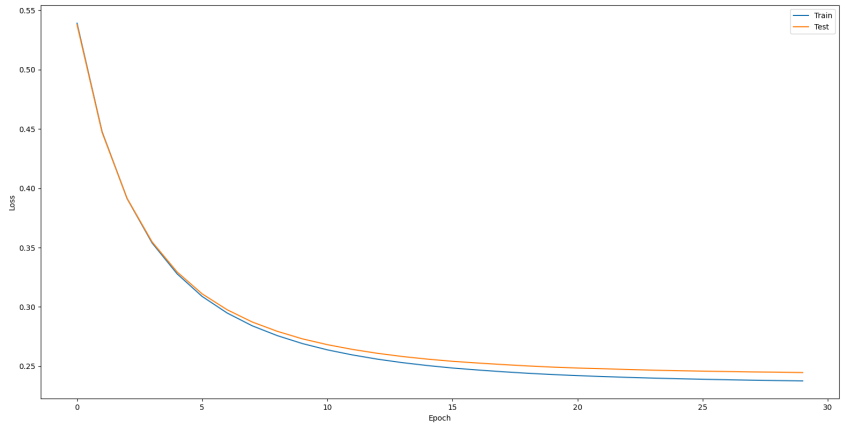


Figure 6: Loss plot for gradient descent learning rate of 0.1.

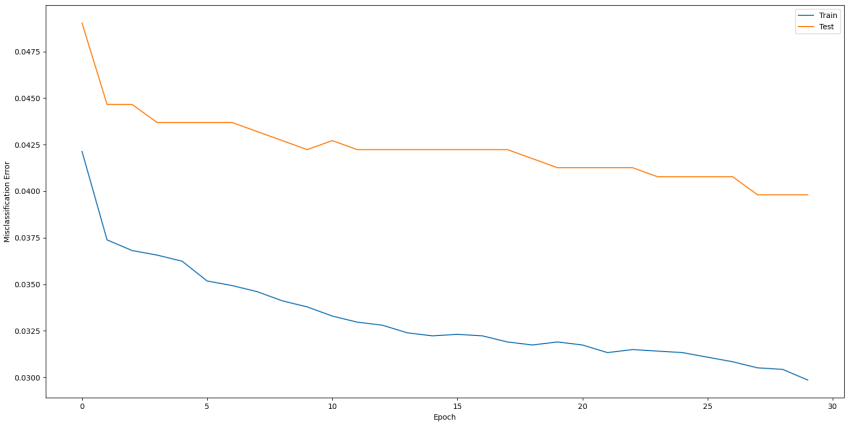


Figure 7: Misclassification plot for gradient descent learning rate of 0.1.

c. *[7 points]* Repeat (b) using stochastic gradient descent with a batch size of 1. Note, the expected gradient with respect to the random selection should be equal to the gradient found in part (a). Show both plots described in (b) when using batch size 1. Take careful note of how to scale the regularizer.

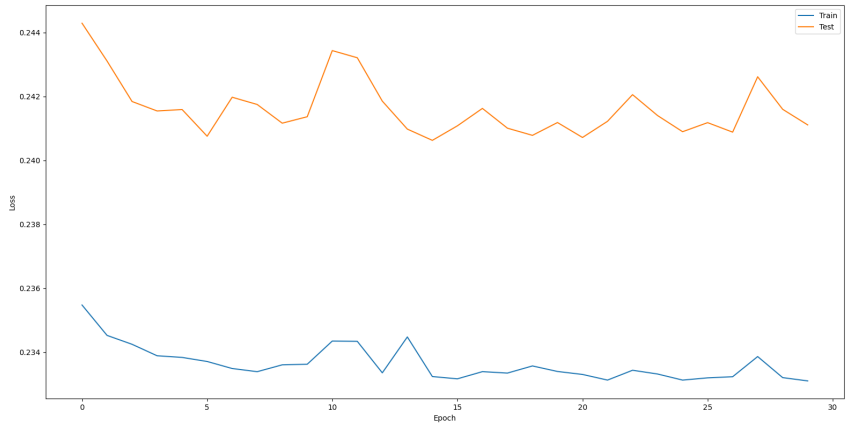


Figure 8: Loss plot for stochastic gradient descent with a batch size of 1 and learning rate of 0.001.

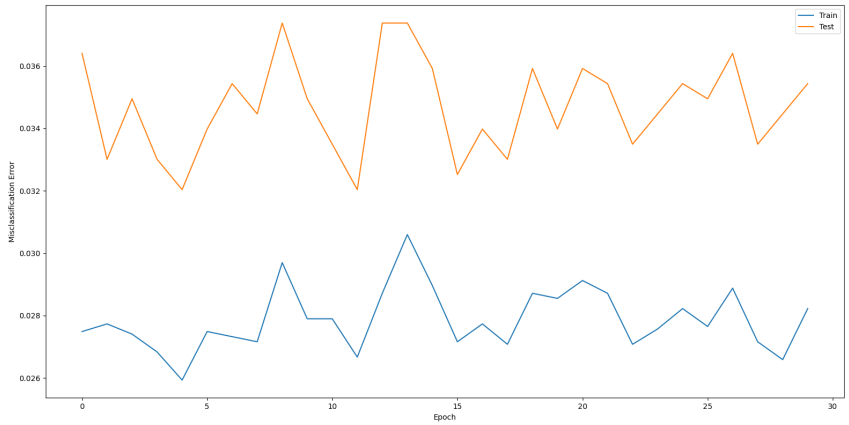


Figure 9: Misclassification plot for stochastic gradient descent with a batch size of 1 and learning rate of 0.001.

d. *[7 points]* Repeat (b) using mini-batch gradient descent with batch size of 100. That is, instead of approximating the gradient with a single example, use 100. Note, the expected gradient with respect to the random selection should be equal to the gradient found in part (a).

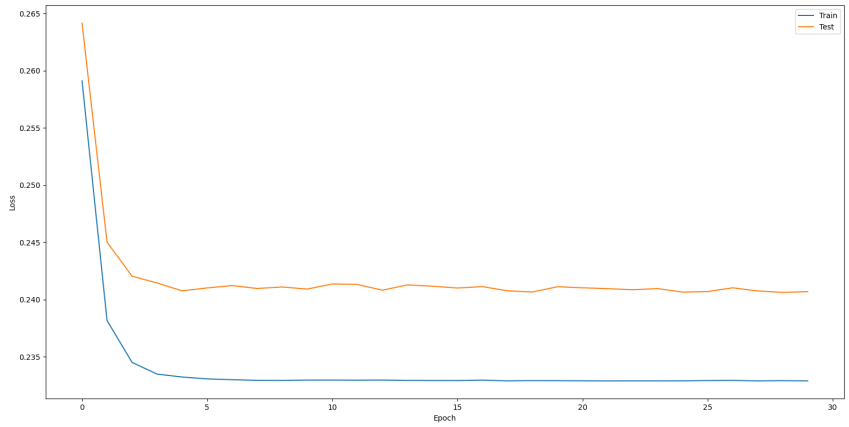


Figure 10: Loss plot for mini-batch gradient descent with a batch size of 100 and learning rate of 0.01.

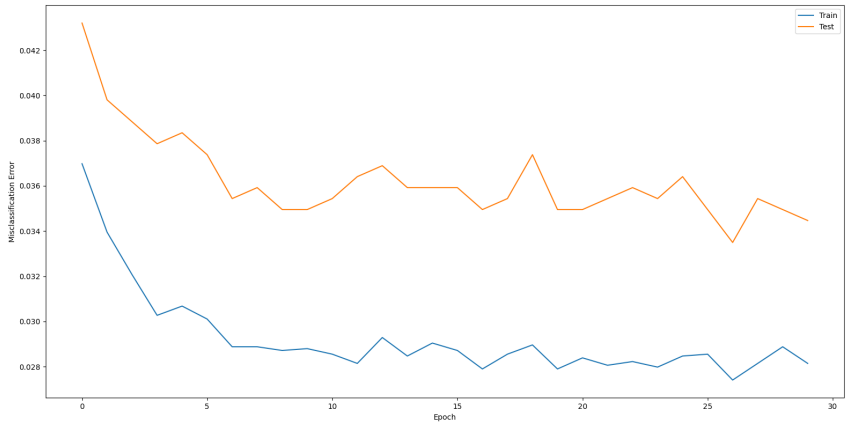


Figure 11: Misclassification plot for mini-batch gradient descent with a batch size of 100 and learning rate of 0.01.

```
class BinaryLogReg:
    @problem.tag("hw2-A", start_line=4)
    def __init__(self, _lambda: float = 1e-3):
        """Initializes the Binary Log Regression model.
        NOTE: Please DO NOT change `self.weight` and `self.bias` values, since it may br

        Args:
            _lambda (float, optional): Ridge Regularization coefficient. Defaults to 1e-
        """
        self._lambda: float = _lambda
        # Fill in with matrix with the correct shape
        self.weight: np.ndarray = None  # type: ignore
        self.bias: float = 0.0

    @problem.tag("hw2-A")
    def mu(self, X: np.ndarray, y: np.ndarray) -> np.ndarray:
```

```
    """Calculate mu in vectorized form, as described in the problem.
    The equation for i^th element of vector mu is given by:

    $$ \mu_i = 1 / (1 + \exp(-y_i (bias + x_i^T weight))) $$

    Args:
        X (np.ndarray): observations represented as `(n, d)` matrix.
            n is number of observations, d is number of features.
            d = 784 in case of MNIST.
        y (np.ndarray): targets represented as `(n, )` vector.
            n is number of observations.

    Returns:
        np.ndarray: An `(n, )` vector containing mu_i for i^th element.
    """
    return 1/(1 + np.exp(-y * (self.bias + (X @ self.weight))))

@problem.tag("hw2-A")
def loss(self, X: np.ndarray, y: np.ndarray) -> float:
    """Calculate loss J as defined in the problem.

    Args:
        X (np.ndarray): observations represented as `(n, d)` matrix.
            n is number of observations, d is number of features.
            d = 784 in case of MNIST.
        y (np.ndarray): targets represented as `(n, )` vector.
            n is number of observations.

    Returns:
        float: Loss given X, y, self.weight, self.bias and self._lambda
    """
    return (1/y.size) * np.sum(np.log(1 + np.exp(-y*(self.bias + (X @ self.weight)))

@problem.tag("hw2-A")
def gradient_J_weight(self, X: np.ndarray, y: np.ndarray) -> np.ndarray:
    """Calculate gradient of loss J with respect to weight.

    Args:
        X (np.ndarray): observations represented as `(n, d)` matrix.
            n is number of observations, d is number of features.
            d = 784 in case of MNIST.
        y (np.ndarray): targets represented as `(n, )` vector.
            n is number of observations.
    Returns:
        np.ndarray: An `(d, )` vector which represents gradient of loss J with respe
    """
    return (1/y.size) * np.matmul(((self.mu(X, y) - 1) * y), X) + (2 * self._lambda

@problem.tag("hw2-A")
def gradient_J_bias(self, X: np.ndarray, y: np.ndarray) -> float:
    """Calculate gradient of loss J with respect to bias.

    Args:
        X (np.ndarray): observations represented as `(n, d)` matrix.
            n is number of observations, d is number of features.
```

```
            d = 784 in case of MNIST.
        y (np.ndarray): targets represented as `(n, )` vector.
            n is number of observations.

    Returns:
        float: A number that represents gradient of loss J with respect to self.bias
    """
    return (1/y.size) * ((self.mu(X, y) @ y) - np.sum(y))

@problem.tag("hw2-A")
def predict(self, X: np.ndarray) -> np.ndarray:
    """Given X, weight and bias predict values of y.

    Args:
        X (np.ndarray): observations represented as `(n, d)` matrix.
            n is number of observations, d is number of features.
            d = 784 in case of MNIST.

    Returns:
        np.ndarray: An `(n, )` array of either -1s or 1s representing guess for each
    """
    return np.sign((X @ self.weight) + self.bias)

@problem.tag("hw2-A")
def misclassification_error(self, X: np.ndarray, y: np.ndarray) -> float:
    """Calculates misclassification error (the rate at which this model is making in
    Note that `misclassification_error = 1 - accuracy`.

    Args:
        X (np.ndarray): observations represented as `(n, d)` matrix.
            n is number of observations, d is number of features.
            d = 784 in case of MNIST.
        y (np.ndarray): targets represented as `(n, )` vector.
            n is number of observations.

    Returns:
        float: percentage of times prediction did not match target, given an observa
    """
    return np.where(y != self.predict(X))[0].size/y.size

@problem.tag("hw2-A")
def step(self, X: np.ndarray, y: np.ndarray, learning_rate: float = 1e-4):
    """Single step in training loop.
    It does not return anything but should update self.weight and self.bias with co

    Args:
        X (np.ndarray): observations represented as `(n, d)` matrix.
            n is number of observations, d is number of features.
            d = 784 in case of MNIST.
        y (np.ndarray): targets represented as `(n, )` vector.
            n is number of observations.
        learning_rate (float, optional): Learning rate of SGD/GD algorithm.
            Defaults to 1e-4.
    """
    self.weight = self.weight - self.gradient_J_weight(X, y) * learning_rate
```

```
        self.bias = self.bias - self.gradient_J_bias(X, y) * learning_rate

    @problem.tag("hw2-A", start_line=7)
    def train(
        self,
        X_train: np.ndarray,
        y_train: np.ndarray,
        X_test: np.ndarray,
        y_test: np.ndarray,
        learning_rate: float = 1e-2,
        epochs: int = 30,
        batch_size: int = 100,
    ) -> Dict[str, List[float]]:
        """Train function that given dataset X_train and y_train adjusts weights and bia
        It also should calculate misclassification error and J loss at the END of each e

        For each epoch please call step function `num_batches` times as defined on top o

        NOTE: This function due to complexity and number of possible implementations wi
        However, we might still test it using gradescope, and you will be graded based o

        Args:
            X_train (np.ndarray): observations in training set represented as `(n, d)` m
                n is number of observations, d is number of features.
                d = 784 in case of MNIST.
            y_train (np.ndarray): targets in training set represented as `(n, )` vector.
                n is number of observations.
            X_test (np.ndarray): observations in testing set represented as `(m, d)` ma
                m is number of observations, d is number of features.
                d = 784 in case of MNIST.
            y_test (np.ndarray): targets in testing set represented as `(m, )` vector.
                m is number of observations.
            learning_rate (float, optional): Learning rate of SGD/GD algorithm. Defaults
            epochs (int, optional): Number of epochs (loops through the whole data) to u
                Defaults to 30.
            batch_size (int, optional): Number of observation/target pairs to use for a
                Defaults to 100.

        Returns:
            Dict[str, List[float]]: Dictionary containing 4 keys, each pointing to a lis
            {
                "training_losses": [<Loss at the end of each epoch on training set>],
                "training_errors": [<Misclassification error at the end of each epoch on
                "testing_losses": [<Same as above but for testing set>],
                "testing_errors": [<Same as above but for testing set>],
            }
            Skeleton for this result is provided in the starter code.

        Note:
            - When shuffling batches/randomly choosing batches makes sure you are using
        """
        num_batches = int(np.ceil(len(X_train) // batch_size))
        result: Dict[str, List[float]] = {
            "train_losses": [],  # You should append to these lists
            "train_errors": [],
```

```
            "test_losses": [],
            "test_errors": [],
        }
        self.weight = np.zeros(X_train.shape[1])
        for i in range(epochs):
            for j in range(num_batches):
                argBatch = RNG.choice(np.arange(X_train.shape[0]), size=batch_size)
                self.step(X_train[argBatch, :], y_train[argBatch], learning_rate=learnin

            result["train_losses"].append(self.loss(X_train, y_train))
            result["train_errors"].append(self.misclassification_error(X_train, y_train)
            result["test_losses"].append(self.loss(X_test, y_test))
            result["test_errors"].append(self.misclassification_error(X_test, y_test))

        return result


if __name__ == "__main__":
    model = BinaryLogReg(_lambda = 0.1)
    (x_train, y_train), (x_test, y_test) = load_2_7_mnist()

    GD_trained_model = model.train(x_train, y_train, x_test, y_test, learning_rate=0.1,

    # Plot losses
    plt.plot(GD_trained_model["train_losses"], label="Train")
    plt.plot(GD_trained_model["test_losses"], label="Test")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend()
    plt.show()

    # Plot error
    plt.plot(GD_trained_model["train_errors"], label="Train")
    plt.plot(GD_trained_model["test_errors"], label="Test")
    plt.xlabel("Epoch")
    plt.ylabel("Misclassification_Error")
    plt.legend()
    plt.show()

    trained_model_batch1 = model.train(x_train, y_train, x_test, y_test, learning_rate=

    # Plot losses
    plt.plot(trained_model_batch1["train_losses"], label="Train")
    plt.plot(trained_model_batch1["test_losses"], label="Test")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend()
    plt.show()

    # Plot error
    plt.plot(trained_model_batch1["train_errors"], label="Train")
    plt.plot(trained_model_batch1["test_errors"], label="Test")
    plt.xlabel("Epoch")
    plt.ylabel("Misclassification_Error")
    plt.legend()
```

```
plt.show()

trained_model_batch100 = model.train(x_train, y_train, x_test, y_test, learning_rate

# Plot losses
plt.plot(trained_model_batch100["train_losses"], label="Train")
plt.plot(trained_model_batch100["test_losses"], label="Test")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

# Plot error
plt.plot(trained_model_batch100["train_errors"], label="Train")
plt.plot(trained_model_batch100["test_errors"], label="Test")
plt.xlabel("Epoch")
plt.ylabel("Misclassification Error")
plt.legend()
plt.show()
```

Administrative

A8.

a. *[2 points]* About how many hours did you spend on this homework? There is no right or wrong answer :)

I spent about 20 hours on this homework.